SecureShield: Role-Based Access Control (RBAC) API

Mini Project II: SecureShield A Python-based API built with Flask that simulates a secure backend system. This project implements a robust authentication flow using JSON Web Tokens (JWT) and a Role-Based Access Control (RBAC) mechanism prioritizing the Principle of Least Privilege.

Implemented Features

Part 1: Authentication & Identity

- **Task 1: Secure Password Storage:** Implemented a registration system where passwords are cryptographically salted and hashed using `flask-bcrypt` before being saved to the local JSON database. Plain text passwords are never stored.
- **Task 2: JWT Issuance:** Built a `/login` endpoint that authenticates users and generates a JWT payload containing the user's `username`, `role`, and expiration timestamp.
- **Task 3: Token Validation:** Developed a custom `@token_required` middleware that intercepts requests to protected routes, validating the presence, integrity, and expiration of the JWT in the `Authorization` header.

Part 2: Access Control & Authorization

- **Task 4: Role-Based Routing:** Configured strict access levels:
  - `GET /profile`: Accessible by both **User** and **Admin** roles.
  - `DELETE /user/<id>`: Restricted completely; accessible only by the **Admin** role.
- **Task 5: Token Revocation (Blacklisting):** Implemented a `/logout` endpoint that captures the active JWT and stores it in a blacklist database, successfully invalidating the token before its natural expiration.
- **Task 6: Defensive Logging:** Integrated a security logger that writes every unauthorized access attempt (403 Forbidden) to `security.log`, capturing the timestamp, username, and the restricted route they attempted to access.

Technology Stack

- **Framework:** Flask (Python)
- **Authentication:** PyJWT
- **Cryptography:** Flask-Bcrypt
- **Data Storage:** Local JSON (`users.json`, `blacklist.json`)

Installation & Setup

1. **Clone the repository:**

```
git clone [https://github.com/YOUR_GITHUB_USERNAME/SecureShield-RBAC-API.git](h
cd SecureShield-RBAC-API
```

2. **Install the required dependencies:**

```
pip install -r requirements.txt
```

3. **Initialize the server:**

```
python secure_auth_api_fixed.py
```

The API will be locally hosted at `http://127.0.0.1:5000`

Security Report

The theoretical answers regarding cryptography and token security are available in the attached `report.pdf` document. It addresses:

1. The mathematical necessity of salting to prevent Rainbow Table attacks.
2. The security risks associated with storing sensitive plaintext data within a Base64 encoded JWT payload.